COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

2026 Bc. Filip Sršeň

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Ján Kľuka, PhD.

Bratislava, 2026

Bc. Filip Sršeň

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Filip Sršeň
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: A Modular Cluster for AI Research in Resource-Constrained Academic Environments
Modulárny klaster pre výskum v AI v akademickom prostredí s obmedzenými zdrojmi

Anotácia: Významná časť výskumu na našej katedre sa opiera o metódy strojového učenia, no dostupná výpočtová infraštruktúra je obmedzená a náročná na správu. Výskumníci pravidelne narážajú na problémy so zdieľaním hardvérových zdrojov, efektívnou prácou s dátovými súbormi a presunom experimentov medzi rôznymi prostrediami. Súčasné postupy – ako manuálne plánovanie, udržiavanie samostatných softvérových inštalácií či kopírovanie dát medzi servermi – komplikujú prácu a znižujú jej produktivitu.

Táto práca sa zameriava na riešenie týchto výziev skúmaním možností návrhu modulárneho klastra prispôsobeného potrebám katedry využitím súčasných technológií na virtualizáciu, kontajnerizáciu a správu klastrov implementovaných ako slobodný softvér/softvér s otvoreným zdrojovým kódom (Proxmox, Docker, Kubernetes, SLURM a pod.). Zámerom je navrhnúť riešenie zlepšujúce prístupnosť, škálovateľnosť a efektivitu práce s ohľadom praktické obmedzenia dostupných zdrojov a pracovné postupy používateľov. Implementácia má demonštrovať, ako tieto môžu uvedené technológie podporovať akademický výskum poskytovaním základu, ktorý uľahčuje spoluprácu, zjednodušuje pracovné postupy a zvyšuje reprodukovateľnosť experimentov strojového učenia.

Cieľ: Hlavným cieľom tejto práce je navrhnúť a implementovať modulárny výpočtový klaster prispôsobený technickým a prevádzkovým obmedzeniam akademického výskumného prostredia. Okrem zabezpečenia zdieľanej výpočtovej kapacity bude klaster slúžiť ako platforma integrujúca do jedného celku plánovanie výpočtových úloh, správu prostredí a sledovanie experimentov. Implementácia systému ukáže, ako možno moderné orchestračné technológie prispôsobiť menšej akademickej infraštruktúre s cieľom zlepšiť reprodukovateľnosť a spoľahlivosť, zjednodušiť údržbu a efektívne využívať dostupné výpočtové zdroje.

Literatúra:
Yoo, A.B., Jette, M.A., Grondona, M (2003). SLURM: Simple Linux Utility for Resource Management. In: JSSPP 2003. Springer.
Domingus, J., Arundel, J. (2022): Cloud Native DevOps with Kubernetes, 2nd Edition. O'Reilly.

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Salatino, M. (2024): Platform Engineering on Kubernetes. Manning.
Spišaková, V., Klusáček, D., Hejtmánek, L. (2023). Using Kubernetes in Academic Environment: Problems and Approaches. In: JSSPP 2022. LNCS 13592. Springer
Tim Wickberg (2025). Slinky: Slurm In Kubernetes, Performant AI And HPC Workload Management. In: KubeCon Europe.
Petrosyan, D.A. (2025). Scheduling ML and HPC Jobs with Shoc Platform over Kubernetes. Phys. Part. Nuclei 56, 1581–1585.
Chazapis, A., Nikolaidis, F., Marazakis, M., Bilas, A. (2023). Running Kubernetes Workloads on HPC. In: ISC High Performance 2023. LNCS 13999. Springer.
Misale, C. et al. (2022). Towards Standard Kubernetes Scheduling Interfaces for Converged Computing. In: SMC 2021. CCIS 1512. Springer.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 31.10.2025

Dátum schválenia: 11.11.2025

prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

. . . . . .

Bratislava, 2026 Bc. Filip Sršeň

Acknowledgement

TODO

Abstract

TODO

Keywords: space debris, machine learning, space object classification

Abstrakt

TODO

Kľúčové slová: vesmírny odpad, strojové učenie, klasifikácia vesmírnych objektov

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.1.1 Computing at our department . . . 1

1.1.2 Current state . . . 2

1.2 Problem statement . . . 2

1.2.1 Data and storage management . . . 2

1.2.2 Environment management . . . 2

1.2.3 Scheduling . . . 3

1.3 Thesis Contributions . . . 3

1.3.1 Research . . . 3

1.3.2 Design . . . 3

1.3.3 Implementation . . . 4

1.4 Thesis Structure . . . 4

2 Background & Related Work **5**

2.1 Cloud-Native Orchestration . . . 5

2.1.1 Containerization Fundamentals . . . 5

2.1.2 Container Orchestration Platforms . . . 6

2.1.3 Kubernetes Scheduling and Resource Management . . . 7

2.1.4 Cloud-Native in Academic and HPC Environments . . . 8

2.2 Traditional HPC Scheduling vs. Cloud-Native Orchestration . . . 9

2.2.1 Traditional HPC: Slurm . . . 9

2.2.2 Cloud-Native: Kubernetes . . . 11

2.2.3 Comparison summary . . . 12

2.3 Batch vs. Interactive Computing . . . 13

2.3.1 Common use-cases . . . 13

2.4 Distributed Machine Learning . . . 14

2.5 State of the Art . . . 14

2.5.1 Technologies in academic practice . . . 14

2.5.2 Hardware requirements . . . 14

3 System Architecture Design **16**
3.1 Introduction 16
3.1.1 Design Objectives 16
3.1.2 Design Philosophy 16
3.2 High-Level System Model 16
3.2.1 Layered Abstraction Framework 16
3.2.2 System Interaction Model 16
3.2.3 The Modular Interconnectivity 16
3.3 Detailed Component Architecture 16
3.3.1 Identity and Governance Layer 16
3.3.2 Research Interface and Environment Provisioning 17
3.3.3 Distributed Orchestration and Scheduling Engine 17
3.3.4 Experiment Lifecycle and Metadata Management 17
3.4 Design for Resource Constraint Mitigation 17
3.4.1 Advanced Scheduling Strategies 17
3.4.2 Elasticity and Resource Elasticity 17
3.5 Summary 17

4 Implementation & Technical Realization **18**
4.1 Deployment Strategy 18
4.2 Implementing the Orchestration Engine 18
4.2.1 Configuring Volcano for Academic Constraints 18
4.2.2 Integrating Kube-Ray for Distributed Scaling 18
4.3 Building the Research Interface 18
4.4 Data and Lifecycle Automation 18

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The widening gap between industry-scale AI compute and the heterogeneous, resource-constrained environments found in academia.

TODO: Expand the opening into a full paragraph motivating why the compute gap matters for academic AI research. Consider citing evidence of the growing hardware demands of modern ML (e.g., model sizes, training costs).

1.1.1 Computing at our department

On a global scale, Comenius University is a rather small institution, with our faculty being an even smaller one, let alone our department. The Department of Applied Informatics has around

TODO: Insert exact number of faculty members.

faculty members and around

TODO: Insert exact number of students.

students. Many of these students are actively doing research in machine learning and adjacent fields, thus requiring computational resources that often go beyond their own personal computers.

The department has its own small server cabinet with a handful of computers equipped with consumer-grade hardware.

TODO: Add specifics about the hardware (GPU models, RAM, storage) — reference the inventory table from the implementation chapter or an appendix.

On request, these machines are available to any student who needs them for their work.

1.1.2 Current state

Currently there are four machines used for compute. Users are provided access to each machine the department manages based on a request from their supervisors. They are also added to a Teams team, which is used to post updates about the machines. It is also used for "reserving" compute slots — users post to the team when, for how long, and on which machine they are going to be computing. Users then use SSH to connect to the selected nodes, move their data to the node, and run their computations.

However, because the computers are standalone machines with no collective management or shared file system beyond user logins, this setup comes with a number of challenges: environment management, data management, and scheduling. These are discussed in detail in the problem statement.

TODO: Add a transition paragraph connecting the current-state description to the problem statement. Briefly preview the three key challenges (data, environment, scheduling) to improve narrative flow.

1.2 Problem statement

The complexity of managing distributed AI workloads and the difficulty of providing a "single-pane-of-glass" interface for researchers without heavy DevOps overhead.

1.2.1 Data and storage management

Machine learning often requires large datasets that need to be present for training and evaluating models. Users transfer their datasets onto the machines manually ahead of computation. The datasets then reside on the machines indefinitely. Two problems stem from this:

- When the machine a user has all their data on is being used by someone else, the user needs to either wait for the other job to finish or move the data elsewhere, causing further strain on the machine.
- Users are often not mindful about deleting their unused data. This causes the file system to fill up and requires periodical clean-ups by the administrator.

1.2.2 Environment management

Users manage their environments, packages, and libraries through Conda. As with data, however, these environments are not shared across the machines. This means that whenever users want to switch machines they need to take their Conda environments

with them. The result is that the same packages are installed multiple times across the machines, increasing file system utilization further.

TODO: Quantify the storage waste if possible — e.g., typical Conda environment size multiplied by number of duplicate installations observed across machines.

1.2.3 Scheduling

Whenever users want to use the machines for compute they are supposed to post into a Teams channel. The post should contain information about their job — mainly the length of the computation and the machine(s) they are selecting. Many times, however, the time estimate is inaccurate, or even worse — users do not post about their job in the channel and just start using the machines. This results in users needing to check each machine individually to confirm whether it is available. There is no centralized monitoring solution, so users manually connect to each node and check its utilization.

TODO: Add a concluding paragraph that transitions to the contribution/design chapters — summarise how the three problems (data, environment, scheduling) motivate the three-part contribution (research, design, implementation).

1.3 Thesis Contributions

This work has three main goals: research, design, and implementation.

1.3.1 Research

The research goal is to conduct an analysis of compute resource usage at our department. This includes describing different use-cases users have for the system in place and creating typical usage scenarios for both users and administrators. User interviews were conducted with students experienced with the current solution, covering their workflows, data, experiment outputs, hardware and software needs, limiting factors, and pain points.

TODO: Forward-reference the chapter/section where interview findings are presented in detail.

1.3.2 Design

The design goal is to design solutions that support the identified use-cases. The solution should:

- be able to exist within the existing infrastructure,

- be easily scalable,
- provide additional services,
- support modern AI/ML practices.

1.3.3 Implementation

After providing solutions that address users' needs, the goal is to select a solution and implement it.

TODO: Briefly state or forward-reference the selection criteria used to choose the final design (e.g., compatibility with existing infrastructure, scalability, operational overhead).

Specifically, this thesis contributes:

- A modular, Kubernetes-native platform for AI research.
- An intelligent scheduling layer for resource-efficient batch processing.
- A distributed executor for scaling research workloads.

TODO: Consider adding a brief closing paragraph that maps each contribution to the thesis structure (Chapter X covers research, Chapter Y covers design, etc.).

1.4 Thesis Structure

Overview of the subsequent chapters.

Chapter 2

Background & Related Work

2.1 Cloud-Native Orchestration

This section introduces the foundational concepts behind cloud-native computing and container orchestration that underpin the system designed in this work. We trace the evolution from monolithic deployments through virtualization to modern container-based platforms, with a focus on the technologies and abstractions that enable efficient resource sharing in multi-user environments.

2.1.1 Containerization Fundamentals

Containerization is an operating-system-level virtualization technique that isolates applications and their dependencies into lightweight, portable units called *containers*. Unlike traditional hardware virtualization, which requires a full guest operating system per virtual machine, containers share the host kernel and use Linux kernel primitives—namely *namespaces* for isolation and *cgroups* for resource limiting—to achieve confinement with minimal overhead [Sultan et al., 2019].

The Docker project, released in 2013, popularized container technology by providing a streamlined workflow for building, distributing, and running container images. Docker introduced a layered filesystem model and a declarative image format that encodes the full dependency stack of an application, ensuring reproducibility across environments. This portability addressed a persistent pain point in scientific computing, where researchers frequently struggled to reproduce experiments due to differences in library versions, system packages, and hardware configurations [Domingus et al., 2022].

The Open Container Initiative (OCI), established in 2015 under the Linux Foundation, standardized the container image format and runtime specification, decoupling the ecosystem from any single vendor. Today, multiple runtimes (runc, crun, containerd, CRI-O) comply with the OCI specification, and registries serve as universal

distribution channels for container images.

For academic computing, containers offer three key advantages over traditional deployment models. First, *reproducibility*: an experiment packaged as a container image produces identical results regardless of the underlying host, eliminating the "works on my machine" problem. Second, *isolation*: cgroups and namespaces prevent workloads from interfering with one another, enabling safe multi-tenancy on shared infrastructure. Third, *portability*: the same container image can run on a researcher's laptop, a department server, or a cloud provider without modification.

TODO: Add a figure showing the container architecture (namespaces, cgroups, layered filesystem) or a comparison diagram VM vs. container.

2.1.2 Container Orchestration Platforms

As container adoption grew beyond single-host deployments, the need for automated management of container lifecycles across clusters of machines became apparent. Container orchestration platforms address this need by providing centralized control over deployment, scaling, networking, and fault tolerance of containerized workloads.

Google's internal cluster management systems, Borg and Omega, pioneered many of the concepts that underpin modern orchestration [Verma et al., 2015]. Borg, in production at Google since 2003, runs hundreds of thousands of jobs from many thousands of applications across clusters of up to tens of thousands of machines each. It introduced the notions of declarative job specifications, automated placement based on resource constraints, and self-healing through continuous reconciliation of desired and actual state. Omega extended these ideas with a more modular architecture based on a shared-state scheduling model.

Burns et al. [2016] describe the evolution from Borg through Omega to Kubernetes, an open-source system released by Google in 2014 that has since become the de facto standard for container orchestration. Kubernetes retains Borg's core design principles—declarative configuration, a reconciliation loop, and label-based object selection—while introducing a more extensible API-driven architecture.

The key Kubernetes abstractions relevant to this work include:

- **Pods**, the smallest deployable units, which encapsulate one or more containers sharing network and storage resources on a single node.
- **Deployments** and **StatefulSets**, which manage replica sets of Pods with different guarantees regarding ordering, persistence, and identity.
- **Services**, which provide stable network endpoints for dynamically placed Pods through label selectors.

- **Custom Resource Definitions (CRDs)**, which extend the Kubernetes API with domain-specific objects, enabling the platform to manage workloads beyond traditional microservices [Salatino, 2024].

The Kubernetes control plane consists of several cooperating components: the *API server* exposes the cluster state via a RESTful interface; the *etcd* distributed store persists this state; the *controller manager* reconciles current state with desired state through a collection of control loops; and the *kubelet* agent on each worker node ensures that the containers assigned to it remain running and healthy. This architecture enables both human operators and automated systems to interact with the cluster uniformly through the API.

TODO: Add a diagram of the Kubernetes control plane architecture (API server, etcd, controller manager, scheduler, kubelet, kube-proxy).

2.1.3 Kubernetes Scheduling and Resource Management

The Kubernetes scheduler is responsible for placing Pods onto nodes that satisfy their resource and constraint requirements. The default scheduler operates in two phases: *filtering*, which eliminates nodes that do not meet hard constraints (resource capacity, node selectors, taints and tolerations), and *scoring*, which ranks the remaining nodes using configurable priority functions [Hightower et al., 2019].

Each container in a Pod specifies *resource requests* (the minimum guaranteed amount) and *limits* (the maximum consumable amount) for CPU and memory. The scheduler uses requests during placement to avoid overcommitting nodes, while limits enforce hard boundaries at runtime via cgroups. Based on the relationship between requests and limits, Kubernetes assigns one of three Quality of Service (QoS) classes to each Pod: *Guaranteed* (requests equal limits), *Burstable* (requests less than limits), and *BestEffort* (no requests or limits specified). Under resource pressure, the kernel's OOM killer preferentially evicts BestEffort and then Burstable Pods, protecting Guaranteed workloads [Senjab et al., 2023].

For elastic scaling, Kubernetes provides the Horizontal Pod Autoscaler (HPA), which automatically adjusts the number of Pod replicas based on observed metrics such as CPU utilization or custom application metrics [Nguyen et al., 2020]. The Vertical Pod Autoscaler (VPA) complements HPA by adjusting individual container resource requests, and the Cluster Autoscaler dynamically adds or removes nodes to match aggregate resource demand.

TODO: Add a table summarizing resource requests vs. limits and their QoS class implications with concrete examples (CPU/memory values).

The default scheduler is designed for long-running services with relatively homogeneous resource requirements. Batch, scientific, and AI training workloads, however,

present distinct scheduling challenges: gang scheduling (all-or-nothing placement of co-dependent tasks), fair sharing among users, preemption of lower-priority jobs, and efficient GPU allocation. To address these needs, the Kubernetes scheduling framework was extended with a plugin architecture that allows custom scheduling logic to be injected at well-defined extension points [Misale et al., 2022]. Projects such as Volcano, Yunikorn, and Kube-Ray leverage this extensibility to provide batch and AI-aware scheduling policies, a topic explored further in Section 2.3.

TODO: Expand on Volcano's gang scheduling and queue management in more detail—this is directly used in the implementation (see orchestration.tex). Reference Volcano-specific literature or docs.

2.1.4 Cloud-Native in Academic and HPC Environments

Traditional high-performance computing (HPC) centers have long relied on batch scheduling systems such as SLURM [Yoo et al., 2003], PBS, or LSF to manage resource allocation. These systems excel at scheduling tightly-coupled parallel jobs on dedicated nodes with strict fairness policies, but they lack native support for the container lifecycle management, service discovery, and declarative APIs that characterize the cloud-native paradigm.

Conversely, Kubernetes was originally designed for stateless microservices and web applications, not for the high-throughput, multi-tenant GPU workloads common in academic research. This mismatch creates a gap that several recent efforts have sought to bridge from both directions.

Spišaková et al. [2023] identify the key problems encountered when deploying Kubernetes in an academic environment, including limited GPU sharing, the absence of fair-share scheduling, and difficulties in providing interactive access alongside batch jobs. Their work proposes approaches for adapting Kubernetes abstractions to academic workflows while preserving the flexibility and reproducibility benefits of containers.

From the HPC side, Chazapis et al. [2023] explore running unmodified Kubernetes workloads directly on HPC clusters, proposing a design that integrates with the existing job management system rather than replacing it. This avoids partitioning the cluster into separate cloud and HPC partitions and retains established accounting and scheduling policies. The Slinky project [Wickberg, 2025] takes the complementary approach of embedding SLURM's scheduling capabilities within Kubernetes, providing familiar HPC job submission semantics on top of a cloud-native infrastructure.

Misale et al. [2022] propose standard scheduling interfaces for converged computing, arguing that the Kubernetes scheduling framework's plugin architecture can serve as a unifying layer for both cloud-native and HPC workloads. Their work demonstrates

how batch scheduling semantics—such as gang scheduling, queue management, and job preemption—can be implemented as Kubernetes scheduler plugins without modifying the core control plane.

TODO: Add a comparison table: SLURM vs. Kubernetes feature matrix (scheduling model, container support, GPU management, fair-share, API style, multi-tenancy). This table would help motivate the design choices in the architecture chapter.

These convergence efforts are directly relevant to this thesis. The modular cluster designed in this work must serve both interactive research sessions (notebooks, development environments) and long-running batch jobs (model training, hyperparameter sweeps), a duality that requires drawing on techniques from both the cloud-native and HPC traditions. The platform engineering approach of extending Kubernetes through CRDs and custom controllers, rather than modifying core components, provides the architectural foundation for this integration [Salatino, 2024, Domingus et al., 2022].

TODO: Tie the concluding paragraph more explicitly to the specific design decisions made in this thesis—reference the architecture chapter (e.g. Volcano integration, Kube-Ray for distributed training) once those sections are written.

2.2 Traditional HPC Scheduling vs. Cloud-Native Orchestration

A central design decision for any compute cluster is the choice of workload management paradigm. The two dominant approaches in research computing are traditional HPC schedulers — exemplified by Slurm — and cloud-native container orchestration platforms — exemplified by Kubernetes. This section compares the two paradigms at a general level, highlighting their respective strengths and limitations in the context of AI/ML research workloads.

2.2.1 Traditional HPC: Slurm

Slurm is the de facto standard workload manager in academic and institutional HPC environments. It provides a mature, battle-tested scheduling engine designed for high-throughput batch processing on large clusters.

TODO: Add a citation for Slurm — e.g., the original Slurm paper (Yoo et al., 2003) or SchedMD documentation. Also cite adoption statistics (e.g., TOP500 survey).

Strengths

- **Fair-share scheduling.** Slurm provides sophisticated priority-based scheduling with fair-share policies, allowing administrators to ensure equitable resource distribution across users and groups over time.
- **Efficient resource allocation.** Slurm's job-level resource model (CPU cores, memory, GPUs as discrete allocatable units) is well-suited for tightly-coupled parallel workloads such as MPI jobs, simulations, and large-scale batch processing.
- **Mature ecosystem.** Decades of development have produced a rich toolchain: advanced reservation systems, array jobs, job dependencies, detailed accounting, and extensive administrative tooling.
- **Proven at scale.** Slurm reliably manages clusters ranging from tens to hundreds of thousands of nodes in production environments worldwide.

  TODO: Cite the original Slurm paper or a survey of HPC scheduler adoption to back these claims.

Limitations

- **Environment rigidity.** Slurm assumes a shared file system and a uniform software environment across nodes. Users typically rely on environment modules or manual Conda setups, which are not portable across clusters and can lead to version conflicts.

  TODO: Cite a source on reproducibility challenges in HPC environments (e.g., a Singularity/Apptainer paper or an HPC reproducibility study).

- **Limited support for interactive workloads.** While Slurm can allocate interactive sessions, its design is fundamentally batch-oriented. Long-running interactive services such as Jupyter notebooks, experiment tracking dashboards, or web-based development environments do not fit naturally into the job scheduling model.

  TODO: Cite work on interactive HPC or Jupyter-on-Slurm approaches to strengthen this claim.

- **No native container orchestration.** Slurm can launch containers (e.g., via Singularity/Apptainer), but it does not manage container lifecycles, networking, or service discovery. Containerized services must be managed externally.

  TODO: Cite the Singularity/Apptainer paper since it is mentioned by name.

- **Monolithic scheduling model.** Slurm's scheduler treats jobs as opaque resource requests. It has no awareness of application-level semantics such as distributed training topology, gang-scheduling requirements, or service dependencies.

2.2.2 Cloud-Native: Kubernetes

Kubernetes is a container orchestration platform originally developed for microservice workloads, but increasingly adopted for AI/ML and batch processing through extensions such as Volcano and Kubeflow.

TODO: Cite Kubeflow and Volcano project papers or official documentation here.

Strengths

- **Container-native environment management.** Every workload runs in an isolated, reproducible container image. Dependencies, libraries, and runtime versions are bundled with the application, eliminating environment conflicts and enabling portability.

  TODO: Back the reproducibility claim with a citation (e.g., a study on container-based reproducibility in scientific computing).

- **Service-oriented architecture.** Kubernetes natively supports long-running services (JupyterHub, MLflow, Weights & Biases, dashboards) alongside batch workloads, providing a unified platform for both interactive and non-interactive use.

- **Declarative infrastructure.** Cluster state is defined declaratively through manifests and managed via GitOps workflows, enabling reproducible deployments, version-controlled configuration, and easy cluster reconstruction.

- **Rich ecosystem for ML.** Specialized frameworks such as Kubeflow, Volcano (batch scheduling), Kube-Ray (distributed compute), and MLflow integration provide purpose-built tooling for AI/ML workflows including distributed training, hyperparameter tuning, and experiment tracking.

  TODO: Cite Kube-Ray, Kubeflow, and Volcano individually rather than listing them without references.

- **Extensibility.** Kubernetes' operator pattern and custom resource definitions (CRDs) allow the platform to be extended with domain-specific scheduling semantics without modifying the core system.

TODO: Cite the Kubernetes operator pattern paper or official documentation.

Limitations

- **Operational complexity.** Kubernetes has a steep learning curve and significant operational overhead. Cluster management requires expertise in networking, storage, security, and container registries.

  TODO: Cite a study or blog post quantifying Kubernetes operational complexity.

- **Resource overhead.** The control plane (API server, etcd, scheduler, controller manager) consumes resources that may be significant on small clusters, leaving less headroom for compute workloads.

  TODO: Quantify the control plane resource overhead (e.g., typical memory/CPU footprint of API server + etcd on a small cluster).

- **Weaker fair-share scheduling by default.** Kubernetes' default scheduler lacks the sophisticated fair-share and priority-decay policies that Slurm provides out of the box. These must be added through extensions such as Volcano.

- **Less mature for tightly-coupled parallel jobs.** Kubernetes is not traditionally designed for MPI-style tightly-coupled parallelism. While support exists (e.g., through MPI operators), it is less mature than Slurm's native MPI integration.

  TODO: Cite the MPI Operator project or Kubeflow MPI documentation.

2.2.3 Comparison summary

Neither paradigm is universally superior — the choice depends on workload characteristics and operational requirements. Slurm excels in environments dominated by traditional batch HPC workloads with homogeneous software stacks. Kubernetes excels where workloads are diverse (batch + interactive + services), environments need to be reproducible and portable, and the research community benefits from modern ML tooling.

For AI/ML research environments specifically, the container-native model offers compelling advantages: reproducible environments, native support for interactive development tools, and a growing ecosystem of ML-specific orchestration frameworks.

These benefits must be weighed against the operational complexity and resource overhead that Kubernetes introduces, particularly in resource-constrained academic settings.

TODO: Add a paragraph explicitly stating which paradigm this thesis chose and why, with a forward-reference to the architecture/design chapter.

2.3 Batch vs. Interactive Computing

The challenge of scheduling high-throughput AI jobs alongside interactive researcher notebooks.

TODO: Expand this into a full introductory paragraph explaining the tension between batch and interactive workloads and why it matters for the platform design.

2.3.1 Common use-cases

Through user interviews with students experienced with the current solution, several common use cases were identified:

TODO: Reference the methodology section or appendix where interview details are described (sample size, interview protocol, etc.).

1. **Script-based batch execution.** Accessing the servers via SSH, copying scripts and datasets onto the machine(s), and running the scripts directly.

2. **Third-party experiment management.** Accessing the servers via SSH, running third-party daemons on the servers, and managing experiments from a web interface. This allows users to run distributed scripts, optimize hyper-parameters of the models, and provides useful metrics about the training process.

   TODO: Name the specific third-party tools mentioned in interviews (e.g., Weights & Biases, MLflow, Optuna) to make the use-case concrete.

3. **Remote development environment.** Using the servers as remote development environments. Users SSH onto the servers and work on their research remotely. This centralization is important for them as they can use thin clients, and if they need to run any quick GPU-heavy calculations to validate their work, there is no need to move to another machine.

TODO: Add subsections expanding on the scheduling challenges for each use-case type and how they conflict (e.g., a long-running notebook holding a GPU vs. a batch training job queuing for it). Forward-reference Volcano's queue/priority mechanisms as the proposed solution.

2.4 Distributed Machine Learning

Paradigms of data and model parallelism and the need for specialized orchestrators.

2.5 State of the Art

Review of existing frameworks (e.g., Kubeflow) and identifying the gap (e.g., complexity, lack of modularity for specialized academic clusters).

TODO: Rewrite this introductory paragraph — the old placeholder text is mixed with the new content. Provide a proper introduction that outlines what the SoTA review covers and its structure.

2.5.1 Technologies in academic practice

There is a common trend of using Python for the majority of development in AI research. Many users work with common machine learning frameworks such as TensorFlow or PyTorch.

TODO: Cite adoption statistics for Python/PyTorch/TensorFlow in ML research (e.g., Stack Overflow survey, Papers with Code statistics).

Some users develop their scripts as Jupyter notebooks. Some convert them into pure Python scripts and run them on the machines. Those who use servers as remote development machines run the notebooks directly.

Users also require metrics from their experiments, whether produced by the ML frameworks themselves or by an ML platform such as Weights & Biases.

TODO: Expand with a brief discussion of experiment management platforms — W&B, MLflow, Neptune — and their relevance to the platform design.

TODO: Add a subsection reviewing existing ML platform frameworks (Kubeflow, Polyaxon, Determined AI, Ray) and identifying their gaps for small academic clusters. This is the core of the SoTA section and is currently missing. Include a comparison table of features relevant to academic use-cases.

2.5.2 Hardware requirements

Computation

Almost all users use the machines to train machine learning models. This usually requires GPUs to accelerate the process. Users also want to utilize multiple GPUs to further speed up the process.

TODO: Cite or reference distributed training paradigms (data parallelism, model parallelism) as motivation for multi-GPU support.

Some users — namely those that work with tabular data — want to run ETL pipelines on the data. This is normally a CPU- and memory-heavy task.

Data storage

A significant portion of research is done in computer vision. This use-case is the most resource-intensive, as image or video datasets can reach hundreds of gigabytes depending on the project.

TODO: Add approximate dataset sizes observed in interviews to ground these claims.

More standard machine learning projects require far less data storage — in the range of megabytes to several gigabytes.

There are also users who work with many small files, which can be quite IOps-intensive.

TODO: Expand the SoTA section significantly: review Kubeflow, Volcano, Kube-Ray, and at least 2–3 competing platforms with citations. Include a comparison table of features relevant to academic use-cases. This section is the backbone for justifying the design choices.

Chapter 3

System Architecture Design

3.1 Introduction

3.1.1 Design Objectives

Modularity, Scalability, and Resource Efficiency.

3.1.2 Design Philosophy

Cloud-native, microservice-based approach using Kubernetes.

3.2 High-Level System Model

3.2.1 Layered Abstraction Framework

Identity, Interface, Orchestration, and Lifecycle.

3.2.2 System Interaction Model

End-to-end workflow from auth to distributed training.

3.2.3 The Modular Interconnectivity

Decoupling via standardized interfaces (APIs/K8s CRDs).

3.3 Detailed Component Architecture

3.3.1 Identity and Governance Layer

Keycloak integration for multi-tenancy.

3.3.2 Research Interface and Environment Provisioning

JupyterHub and Harbor (private registry).

3.3.3 Distributed Orchestration and Scheduling Engine

Volcano

Batch scheduling and gang-scheduling for AI workloads.

Kube-Ray

Distributed compute scaling.

3.3.4 Experiment Lifecycle and Metadata Management

MLflow integration for tracking metrics/artifacts.

3.4 Design for Resource Constraint Mitigation

3.4.1 Advanced Scheduling Strategies

Volcano queueing and priority mechanisms.

3.4.2 Elasticity and Resource Elasticity

K8s/Ray dynamic scaling.

3.5 Summary

Concluding synthesis of modular design meets the problem statement.

Chapter 4

Implementation & Technical Realization

4.1 Deployment Strategy

Using GitOps and Helm-based manifests to manage the cluster lifecycle.

4.2 Implementing the Orchestration Engine

4.2.1 Configuring Volcano for Academic Constraints

Priority queues and gang-scheduling.

4.2.2 Integrating Kube-Ray for Distributed Scaling

Distributed compute scaling.

4.3 Building the Research Interface

How the API Gateway and JupyterHub provide the unified interface for the end-user.

4.4 Data and Lifecycle Automation

The integration of Harbor for image stability and MLflow for scientific experiment tracking.

Bibliography

Brendan Burns, Brian Grant, David Oppenheimer, Eric A. Brewer, and John Wilkes. Borg, omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

Rajkumar Buyya, Satish Narayana Srirama, Giuliano Casale, Rodrigo N. Calheiros, Yogesh Simmhan, Bipoš Varghese, Sherali Karaulov, Adel Nadjaran Toosi, Marcos D. Assunção, Saurabh Kumar Garg, Raksha M. Vahid, Mariam Kiran, Emmanuele Santos, Karoč Wilczyński, George Kousiouris, Jia Ruan, Christian Baun, Lars Kolbe, Christian Pichler, Bernhard Kolb, Jörg Domaschka, and Stefan Wesner. A manifesto for future generation cloud computing: Research directions for the next decade. *ACM Computing Surveys*, 51(5):1–38, 2018. doi: 10.1145/3241737.

Antony Chazapis, Fotis Nikolaidis, Manolis Marazakis, and Angelos Bilas. Running Kubernetes workloads on HPC. In *ISC High Performance Workshops*. Springer, 2023. doi: 10.1007/978-3-031-47187-0_12.

John Domingus, Arun Lakshmanan, Justin Armitage, and Andrew C. Munro. *Cloud Native DevOps with Kubernetes*. O'Reilly Media, 2nd edition, 2022.

Paolo Di Francesco, Patricia Lago, and Ivano Malavolta. Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150:77–97, 2019. doi: 10.1016/j.jss.2019.01.001.

Víctor Medel Gracia, Rafael Tolosana-Calasanz, José Ángel Bañares, and Omer F. Rana. Characterising resource management performance in Kubernetes. In *Computers & Electrical Engineering*, volume 68, pages 261–280. Elsevier, 2018. doi: 10.1016/j.compeleceng.2018.03.041.

Kelsey Hightower, Brendan Burns, and Joe Beda. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media, 2nd edition, 2019.

Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3):24–35, 2018. doi: 10.1109/MS.2018.2141039.

Claudia Misale, Daniel J. Milroy, Carlos Eduardo Arango Gutierrez, Maurizio Drocco, Stephen Herbein, Dong H. Ahn, Zvonko Kaiser, and Yoonho Park. Towards standard Kubernetes scheduling interfaces for converged computing. In *Sustained Simulation Performance*, pages 229–245. Springer, 2022. doi: 10.1007/978-3-030-96498-6_18.

Thanh-Tung Nguyen, Yu-Jin Yeom, Taehong Kim, Dae-Heon Park, and Sehan Kim. Horizontal pod autoscaling in Kubernetes for elastic container orchestration. *Sensors*, 20(16):4621, 2020. doi: 10.3390/s20164621.

D.A. Petrosyan. Scheduling ML and HPC jobs with Shoc platform over Kubernetes. *Physics of Particles and Nuclei*, 56:1581–1585, 2025.

Mauricio Salatino. *Platform Engineering on Kubernetes*. Manning Publications, 2024.

Khaldoun Senjab, Sohail Abbas, Naveed Ahmed, and Atta ur Rehman Khan. A survey of Kubernetes scheduling algorithms. *Journal of Cloud Computing*, 12:1–26, 2023. doi: 10.1186/s13677-023-00471-1.

Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan van den Heuvel. The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 143:151–178, 2018. doi: 10.1016/j.jss.2018.09.082.

Viktória Spišaková, Dalibor Klusáček, and Lukáš Hejtmánek. Using Kubernetes in academic environment: Problems and approaches. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, pages 202–222. Springer, 2023. doi: 10.1007/978-3-031-22698-4_12.

Sari Sultan, Imtiaz Ahmad, and Tassos Dimitriou. Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7:52976–52996, 2019. doi: 10.1109/ACCESS.2019.2911732.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pages 1–17. ACM, 2015. doi: 10.1145/2741948.2741964.

Tim Wickberg. Slinky: Slurm in Kubernetes — performant AI and HPC workload management. In *KubeCon Europe*, 2025.

Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, pages 44–60. Springer, 2003. doi: 10.1007/10968987_3.

Zhiheng Zhong and Rajkumar Buyya. A cost-efficient container orchestration strategy in Kubernetes-based cloud computing infrastructures with heterogeneous resources. *ACM Transactions on Internet Technology*, 20(2):1–24, 2020. doi: 10.1145/3378447.